Hallucination Detection in Tool Calling

Dataset and methodology

Prefiltering

The dataset construction process was based on the ToolACE dataset, which was selected due to its focus on tool-calling interactions between language models and external tools. The original dataset contains approximately 11,000 samples. However, during the initial inspection phase, it was observed that only around 7% of the dataset instances contained both valid tool outputs and corresponding final agent responses. Since these two components are essential for

hallucination detection in tool-augmented settings, the experiments were restricted to this subset of the data.

Additional filtering was then applied to remove incomplete or low-information samples. In particular:

- samples without a final agent response following the tool output were discarded;
- very short and excessively long samples were filtered out to simplify preprocessing and annotation.

The following constraints were used:

- 20 characters < tool output length < 3000 characters ;
- 20 characters < final agent response length < 5000 characters.

The resulting filtering process did not significantly affect the overall distribution of response lengths, but substantially improved dataset consistency and quality. Histograms and length distribution statistics are provided in the appendix.

Finally, a subset of 500 samples was randomly selected for further experimentation and annotation. Each final dataset instance consisted of:

- the original user query,
- the tool output,
- the subsequent final agent response.

Initial Dataset Prototype

Before constructing the final dataset pipeline, an initial prototype version of the hallucination dataset was developed to validate the feasibility of the task and evaluate baseline hallucination detectors on a simplified setting.

This first version relied on relatively simple rule-based perturbations and contained 300 clean examples. Several types of hallucinations were introduced:

- numerical substitutions (150 examples)
- sentiment inversions (83 examples)
- manually designed overgeneration templates (300 examples)
- manually designed missing tools requests (300 examples)

For numerical perturbations, fixed transformations were applied, such as:

- replacing a value with its negation,
- substituting numbers with predefined constants.

Sentiment perturbations were generated using manually defined replacement dictionaries, for example:

- “good” → “bad”,
- “win” → “lose”.

Additionally, several handcrafted overgeneration and missing-tool templates were created and appended randomly to the generated responses.

```
OVERGENERATION_SENTENCES = [
    "This trend has remained stable over the past few months.",
    "Experts expect this result to improve significantly next month.",
    "This is considered one of the best options currently available.",
    "The situation is likely to become even more favorable soon.",
    "Users have reported consistently positive outcomes with this option.",
    "This result is also supported by recent independent analysis.",
]


MISSING_TOOL_SENTENCES = [
    "Would you like me to book a flight for you based on this information?",
    "Would you like me to purchase this item for you now?",
    "Would you like me to send this information to your email?",
    "Would you like me to schedule a meeting about this?",
    "Would you like me to reserve a hotel for you?",
    "Would you like me to place an order using your account?",
]
```

Baselines Evaluation

Several baseline models were evaluated on the first version of the dataset. The goal of this stage was not only to measure detection quality, but also to analyze the difficulty of the generated hallucinations and identify weaknesses in the initial dataset construction pipeline.

Two baseline approaches were selected.

1. LettuceDetect

The first evaluated baseline was LettuceDetect, which performs hallucination detection using a fine-tuned BERT-based architecture. For the experiments, the `lettucedect-base` was used as the encoder backbone. The obtained classification results were as follows.

	Sentence-Level Metrics			**Span-Level Metrics**		
Class	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**

Not Hallucinated	0.4755	0.4200	0.4460	0.2282	0.6359	0.3359
Hallucinated	0.7972	0.8311	0.8138			

The span-level evaluation was based on overlap between predicted hallucinated spans and ground-truth annotations One important observation was that the **detector frequently produced hallucinated spans that were substantially larger** than the annotated gold spans. As a result, the overlap-based **precision** became relatively low despite high recall.

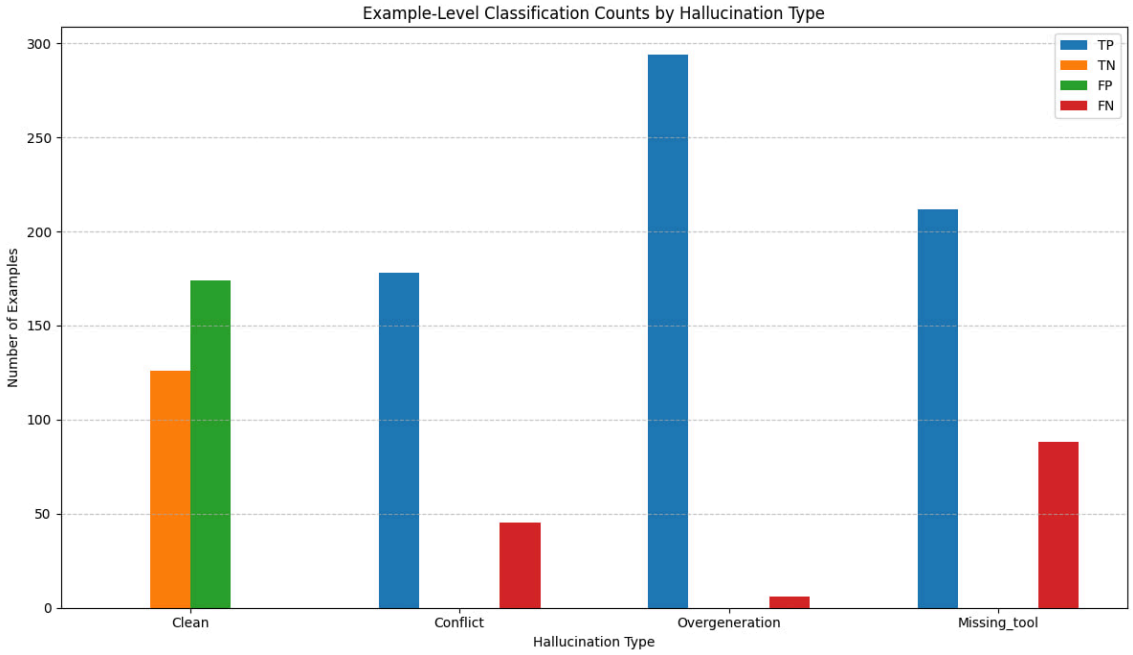
For example, the model produced the following prediction:

Predicted span: "Battle of Hastings in 1206 where he fought"

Ground-truth span: "1206"

This behavior explains the discrepancy between strong hallucination recall and relatively poor span-level precision.

Next, we conducted error analysis by the categories.



The most problematic classification category was the clean (non-hallucinated) class. This behavior is expected because the detector was optimized primarily for hallucination discovery and therefore tended to over-predict suspicious regions.

Among the synthetic hallucination categories, the most **difficult** type was the **missing-tool category**, which contained approximately 90 adversarial examples. Contradiction-based

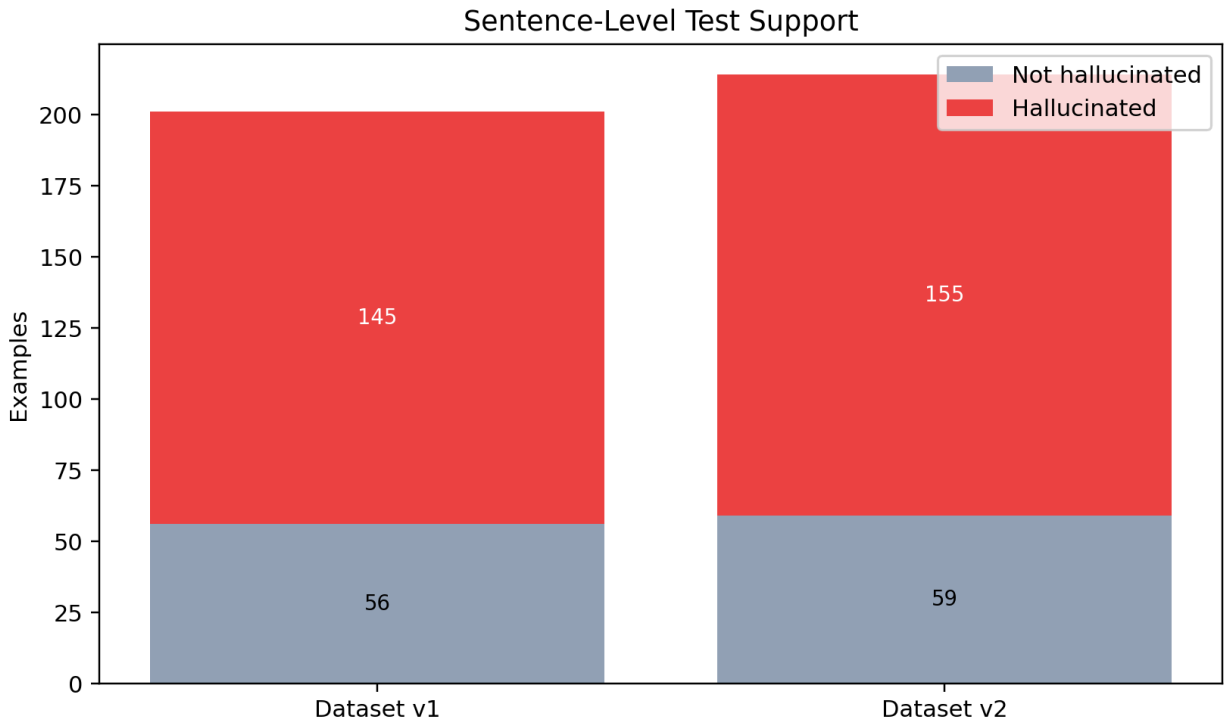
hallucinations contained approximately 45 examples, while the overgeneration category contained only around 5 adversarial examples due to the simplicity of the initial generation pipeline.

2 LookBackLens

Unlike the previous approach, LookBackLens relies on attention-based analysis and evidence tracing mechanisms. The method analyzes internal model attention patterns and compares generated content against supporting evidence retrieved from the provided context.

For these experiments, the **Mistral AI Mistral** model was used as the underlying language model backbone. During preprocessing 46 samples were skipped due to truncation or input length limitations. The final split contained 868 training samples and 209 test samples.

	Sentence-Level Metrics			**Span-Level Metrics**		
Class	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**
Not Hallucinated	0.5082	0.5536	0.5299	0.5909	0.7717	0.6693
Hallucinated	0.8311	0.8039	0.8173			



Compared to the LETUSDetect baseline, LookBackLens achieved substantially **better span-level precision** while maintaining strong hallucination recall.

The strong baseline performance suggested that the initial dataset version was relatively simple for modern hallucination detectors. This motivated the development of a more advanced second-generation pipeline incorporating diverse perturbation strategies, semantically consistent entity substitutions, and LLM-generated overgeneration samples.

Final Dataset

The link to the final dataset: https://huggingface.co/datasets/annnettte/HalluToolACE

Entity Collection and Preprocessing

The first preprocessing stage of the second-generation pipeline consisted of automatic entity collection.

A **rule-based extraction system** was applied to all dataset texts using predefined groups of regular expressions and lexical patterns. The goal of this stage was to identify semantic entities that could later be modified during hallucination generation.

The extracted entity categories included:

- countries/cities/continents,
- company names,

- currencies,
- personal names

All detected entities were stored together with their semantic category information and later used during controlled perturbation generation.

Hallucination Generation Pipeline

Contradiction-Based Hallucinations

The contradiction dataset was generated through controlled perturbations in original tools outputs applied directly to the corresponding agent responses. Several perturbation strategies were implemented:

- date modifications;
- numerical perturbations;
- entity substitutions;
- sentiment inversions.

Date perturbations were generated by applying random temporal offsets to existing dates. Numerical perturbations used multiple stochastic strategies, including: multiplication, division, additive shifts, scaling, digit replacement. One strategy was selected uniformly at random for each modified number.

Entity substitutions were performed by replacing an entity with another entity sampled uniformly from the same semantic category. For example, city names were replaced with other cities, and cryptocurrencies with other cryptocurrencies.

Sentiment perturbations relied on manually constructed antonym dictionaries, as before.

The perturbation pipeline processed all samples sequentially and attempted each transformation type in a predefined order. If a valid perturbation could be applied, the corresponding hallucinated example was generated automatically. Throughout development, manual inspection was repeatedly used to refine regex patterns and perturbation rules. For example, early versions of the numerical extraction system incorrectly handled decimal numbers containing commas instead of periods.

Overgeneration Hallucinations

For this stage, the **Qwen 2.5 1B model** was used to generate synthetic excessive information. The original prompt can be found in the **Appendix B**. The model was prompted to:

- analyze the tool output,
- analyze the original agent response,
- append additional information related to the topic,
- while ensuring that the appended information was unsupported by the tool output.

Multiple retries were used during generation to improve sample quality and reduce malformed outputs. 300 overgeneration examples were generated using this procedure.

Missing-Tool Hallucinations

Several candidate strategies were explored experimentally. After empirical evaluation, five manually designed missing-tool patterns demonstrated the highest quality and were therefore selected for the final dataset version.

Final Baseline Evaluation

1. LettuceDetect

	Sentence-Level Metrics			**Span-Level Metrics**		
Class	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**
Not Hallucinated	0.3684	0.4200	0.3925	0.175	0.452	0.252
Hallucinated	0.7972	**0.7600**	0.7782			

2. LookBackLens

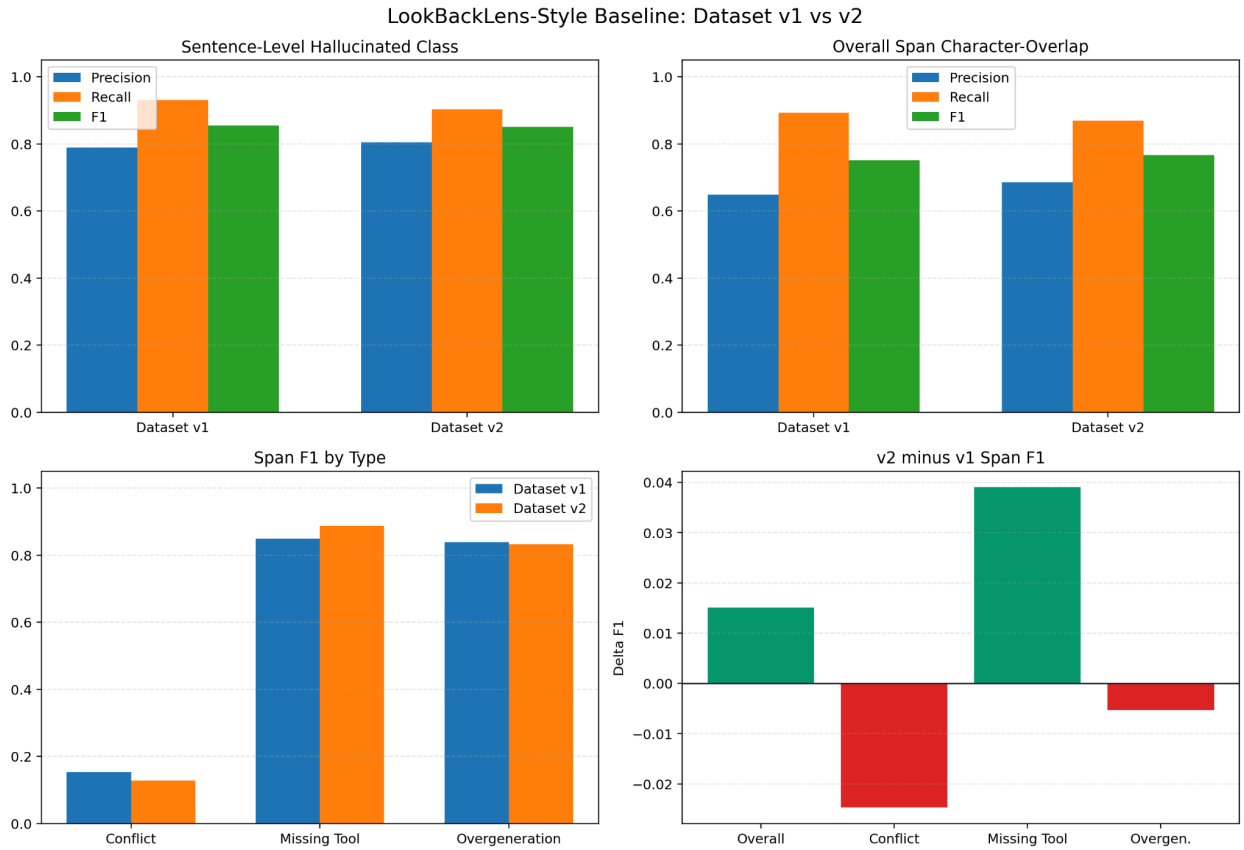
	Sentence-Level Metrics			**Span-Level Metrics**		
Class	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**
Not Hallucinated	0.625	0.437	0.505	0.686	0.868	0.767
Hallucinated	0.805	0.903	0.851			

Since LookBackLens is trained on the HalluToolAce dataset and uses more complex cross-attention patterns detections, it achieves superior quality both on **s**entence-level and span-level metrics

3. LettuceDetect vs LookBackLens Span-Level by Category

Lettuce Detect			
Class	**Precision**	**Recall**	**F1**
Clean	0.0	0.0	0.0
Conflict	0.041	**0.524**	0.076
Tool	0.200	0.293	0.238
Overgen	0.352	0.571	0.435

LookBackLens			
Class	**Precision**	**Recall**	**F1**
Clean	0.0	0.0	0.0
Conflict	0.09	0.224	0.128
Tool	**0.83**	**0.954**	**0.888**
Overgen	**0.804**	**0.864**	**0.833**



On Clean inputs, both systems achieve zero scores for the positive class. For Conflict cases,

performance varies more noticeably. Lettuce Detect shows low precision, but great recall, since the model tends to highlight several words around the injection as hallucinations and finds most hallucinations correctly, while LookBackLens shows worse results. For the Tool class**,** LookBackLens dramatically outperforms Lettuce Detect, with an F1 of 0.888 vs. 0.238 — a more than 3.7 improvement. For the Overgen class**,** LookBackLens again leads substantially, achieving F1 = 0.833 compared to Lettuce Detect's 0.435. This indicates that LookBackLens provides more reliable and balanced detection, whereas Lettuce Detect tends toward over-prediction around injection regions.

Experiments

Lettuce Detect Fine-Tune

The dataset is split **70 / 15 / 15**, stratified by hallucination type to balance class distribution. The test set is held out and used only for final evaluation. LettuceDetect was fine-tuned for **5 epochs**.

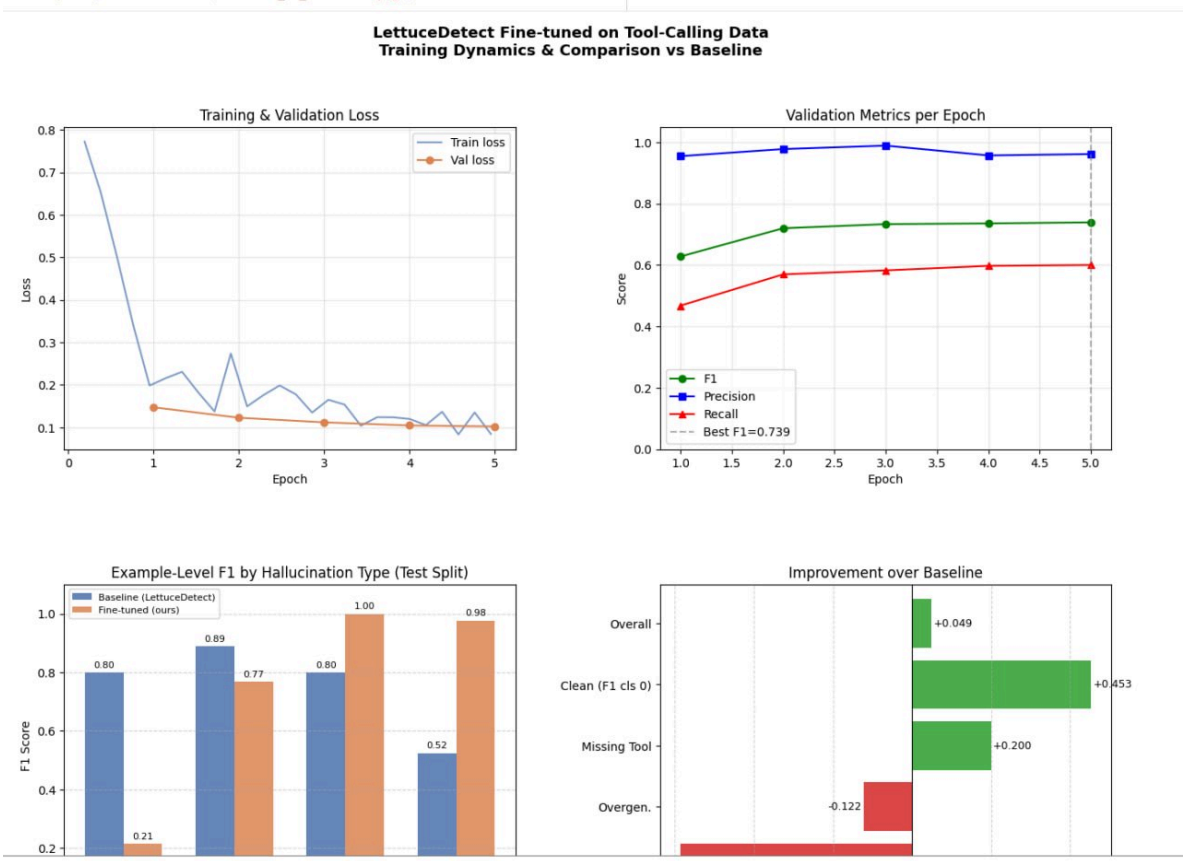
- Loss curves**:** Training loss drops from ~0.77 to ~0.10; validation loss declines smoothly and tracks training loss closely — no overfitting.
- Validation metrics**:** Precision stays high (~0.95–0.99), recall grows steadily (0.46 → 0.60), and F1 peaks at 0.739 (epoch 5), selected as the final checkpoint.

The pattern reflects a conservative but increasingly sensitive detector — ideal for hallucination detection.

Test-Set Results vs. Baseline

Type	Baseline F1	Fine-tuned F1	Delta
conflict	0.8000	0.2143	-0.5857
overgeneration	0.8889	0.7671	-0.1218
missing_tool	0.8000	1.0000	**+0.2000**
clean	0.5246	0.9773	**+0.4527**
overall	0.7385	0.7876	**+0.0491**

The Missing Tool and Clean classification reach almost perfect scores, while the conflict cases see a substantial decline. Overall, training is stable, well-**c**onverged, and yields strong domain-specific improvements on tool-calling hallucinations.



Hybrid

To further analyze the strengths of both approaches, a hybrid evaluation strategy was introduced. For each hallucination type, the model with the higher validation performance was selected: either the original baseline detector or the fine-tuned model. The final hybrid system therefore combined predictions from different models depending on the hallucination category.

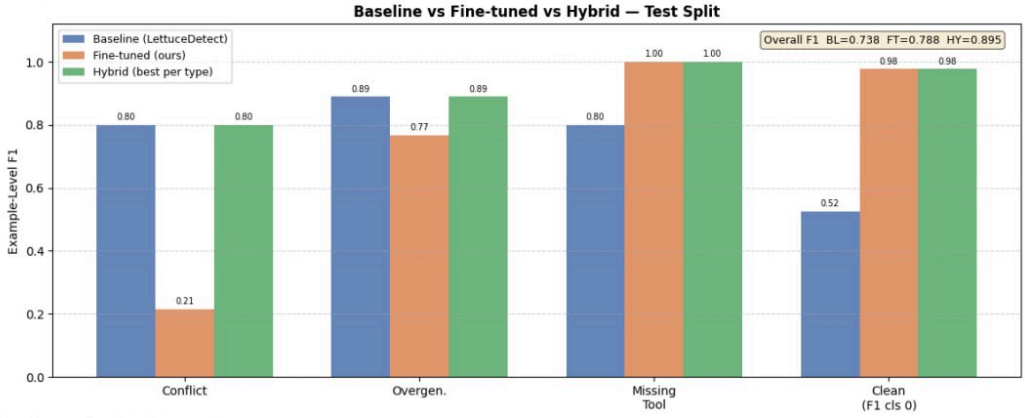
For each hallucination type, example-level F1 scores were computed separately for the baseline, the fine-tuned model, and the hybrid configuration. The evaluation was performed on the test split, where an example was considered hallucinated if at least one hallucinated token was predicted.

The results demonstrated that the hybrid approach consistently preserved the strengths of both models and achieved the best overall balance across hallucination types.

```
79  print('Saved → results/hybrid_comparison.png')
```

```
Baseline types: ['conflict', 'overgeneration']
Fine-tuned types: ['missing_tool', 'clean']

Type                 Baseline    Fine-tuned    Hybrid   Best
------------------------------------------------------------
conflict               0.8000        0.2143    0.8000     BL
overgeneration         0.8889        0.7671    0.8889     BL
missing_tool           0.8000        1.0000    1.0000     FT
clean *                0.5246        0.9773    0.9773     FT
overall                0.7385        0.7876    0.8952     HY
```




```
Saved → results/hybrid_comparison.png
```

LettuceDetect Fine-Tune with Limited Clear Data

To investigate whether the large number of clean examples biased the model toward predicting non-hallucinated spans, an additional fine-tuning experiment was conducted using a reduced amount of clean training data. After splitting the dataset into training, validation, and test sets, the number of clean examples in the training split was reduced by **half**, while the validation and test sets remained unchanged. The model was then fine-tuned on this modified training set and evaluated on the original full test set.

The motivation behind this experiment was the hypothesis that overexposure to clean examples may encourage the model to memorize clean patterns and become overly conservative when predicting hallucinations. This could potentially explain the particularly weak performance on the Conflict category observed during previous experiments.

Type	Baseline F1	Fine-tuned F1	Delta
conflict	0.8000	0.2069	-0.5931
overgeneration	0.8889	0.7671	-0.1218
missing_tool	0.8000	1.0000	**+0.2000**
clean	0.5246	0.9535	**+0.4289**
overall	0.7385	0.7755	**+0.0370**

However, the hypothesis was not confirmed. The performance on Conflict examples still improved substantially after fine-tuning, while the performance on Clean examples increased less significantly due to the reduced amount of clean training data. These findings suggested that the issue was not simply caused by class imbalance or memorization of clean patterns, motivating a more targeted approach focused directly on Conflict examples.

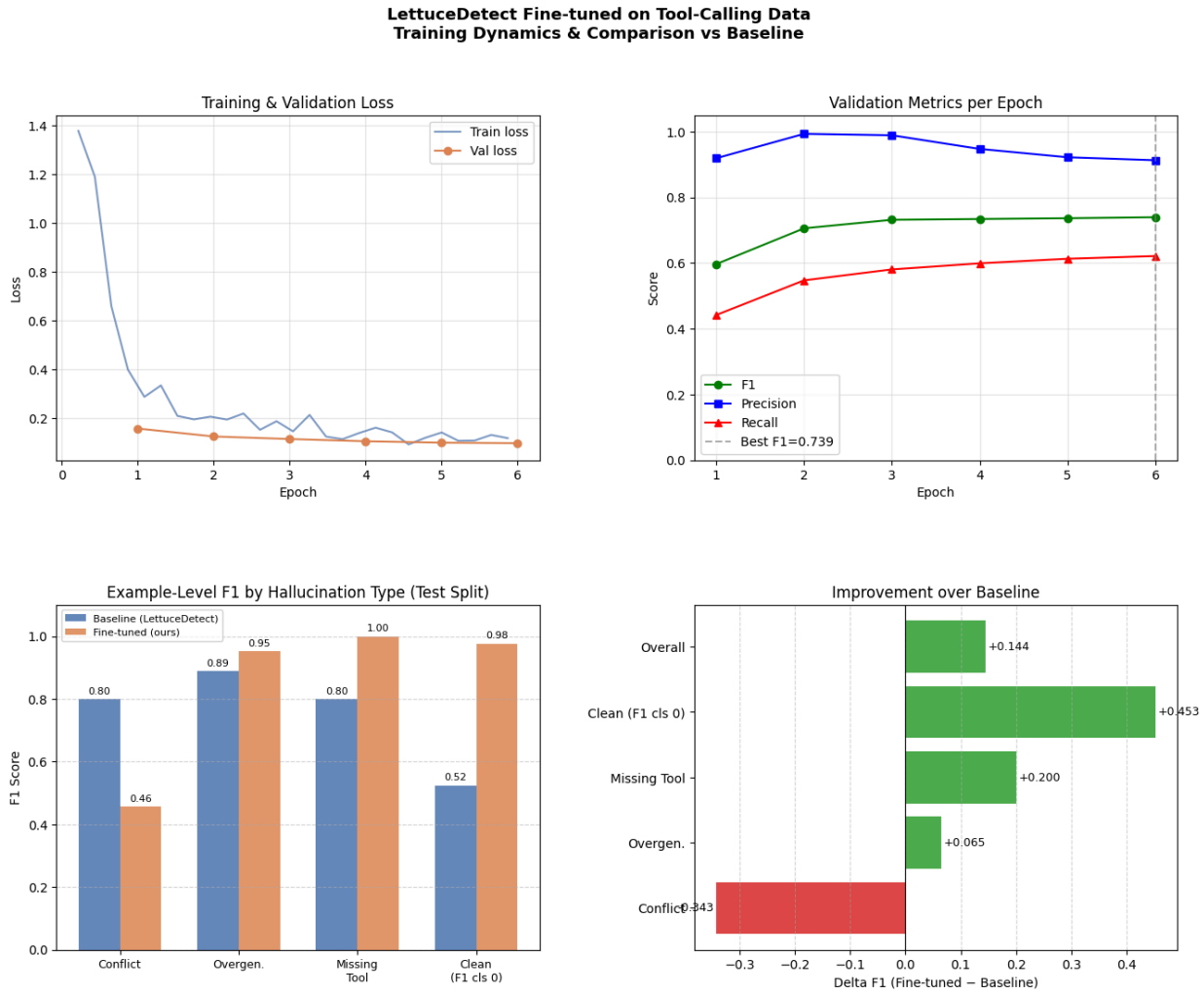
LettuceDetect Fine-Tune with Importance Conflict Data

Based on the previous findings, a second experiment introduced explicit weighting for Conflict examples during training. In this setup, the full original training dataset was restored. All training examples were assigned a default weight of 1.0, while examples belonging to the Conflict category were assigned an **increased weight of 1.5.** The model was then fine-tuned for five epochs using this weighted loss formulation.

Unlike the previous experiment, this approach directly emphasized Conflict examples during optimization. The results showed improvements across all evaluation metrics, while the degradation on Conflict examples became substantially smaller compared to earlier fine-tuning runs. Therefore, this approach can be considered successful.

Type	Baseline F1	Fine-tuned F1	Delta
conflict	0.8000	0.4571	-0.3429
overgeneration	0.8889	0.9535	**+0.0646**
missing_tool	0.8000	1.0000	**+0.2000**
clean	0.5246	0.9773	**+0.4527**
overall	0.7385	0.8826	**+0.1442**

In particular, while the previous fine-tuning setup achieved an F1 score of only 0.005 on the Conflict subset, the weighted training approach improved the F1 score to **0.14**, representing a substantial improvement in the model's ability to detect hallucinations in Conflict examples.



Fine-Tune DeBERTa-v3-large

We also tried to evaluate microsoft/deberta-v3-base as a drop-in replacement for the ModernBERT-based LettuceDetect backbone, keeping the same two-class token classification head and layer-freezing strategy. The model consistently produced F1 = 0 across all training epochs. We attribute this to DeBERTa-v3-base's 512-token limit: our tool-call contexts are often longer, so the model sees only a truncated input and loses the context it needs to detect hallucinations. Without that context, training collapses to predicting the majority class.

Epoch	Training Loss	Validation Loss	Precision	Recall	F1
1	1.624518	0.894996	0.000000	0.000000	0.000000
2	1.469721	0.647691	0.000000	0.000000	0.000000

Postprocessing steps

We additionally explored simple post-processing heuristics aimed at improving hallucination detection in cases where the model initially predicts a sample as non-hallucinated. The main idea was to automatically verify whether important information from the tool output is reflected in the final model response.

In particular, we experimented with rule-based matching using regular expressions. For example, numerical values appearing in the tool output were searched for in the generated response, optionally together with nearby keywords. Similar heuristics could also be applied to sentiment cues or entities extracted from the tool output.

However, this approach has several limitations. In many cases, tool outputs contain numerous numerical values or entities that are not semantically important for the final answer. As a result, naive matching may both miss true hallucinations and introduce false positives. In our preliminary experiments on conflict examples, simple numeric matching slightly improved detection performance, reducing the number of observed errors from approximately 35 to 31, although several previously correct cases became misclassified.

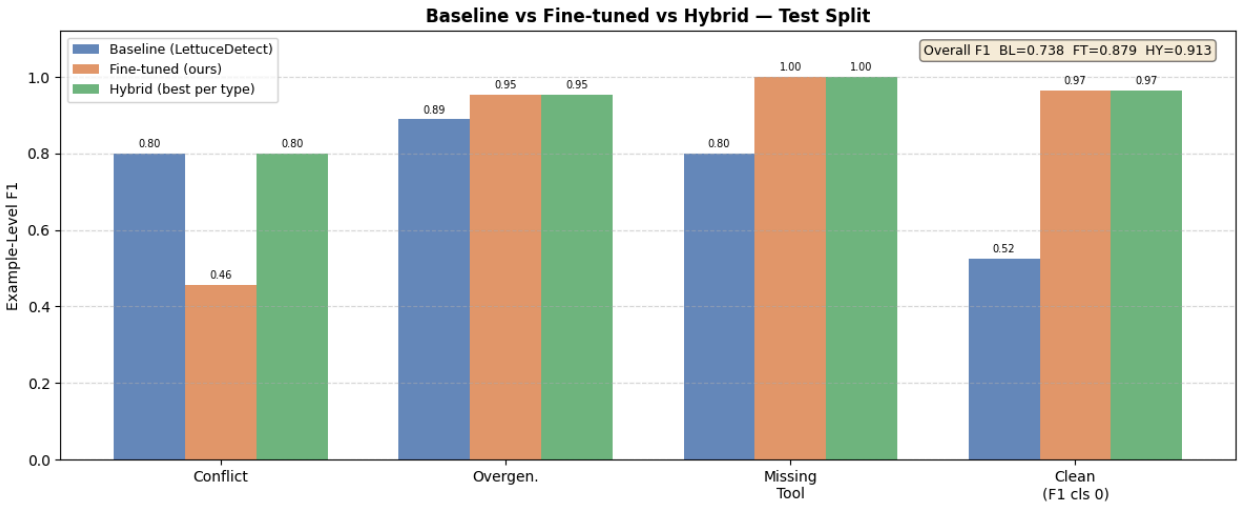
Overall, these experiments suggest that lightweight post-processing heuristics can provide modest gains, particularly for conflict-style hallucinations. A more promising direction would be to extract such signals as structured features and integrate them into a trainable classifier rather than relying on manually designed rules alone

Final Hybrid Approach

Type	Baseline	Fine-tuned	Hybrid	Best
conflict	**0.8000**	0.4571	0.8000	BL
overgeneration	0.8889	**0.9535**	0.9535	FT
missing_tool	0.8000	**1.0000**	1.0000	FT
clean	0.5246	**0.9655**	0.9655	FT
overall	0.7385	0.8785	**0.9134**	HY

The Hybrid approach shows further improvement.

Model's weights can be found here: https://huggingface.co/annnettte/lettucedect-tool-calling

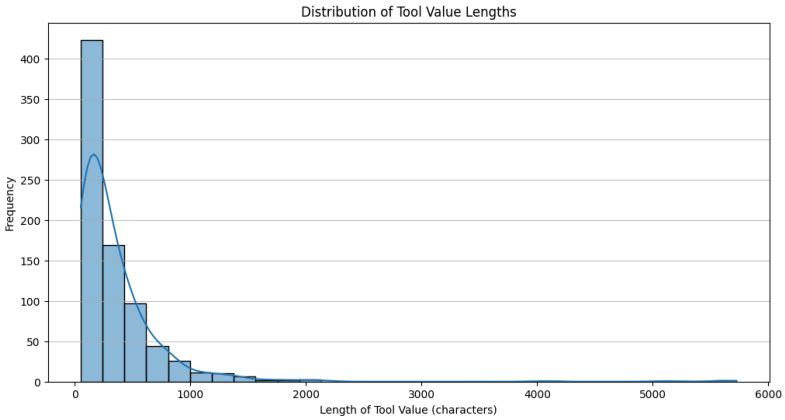


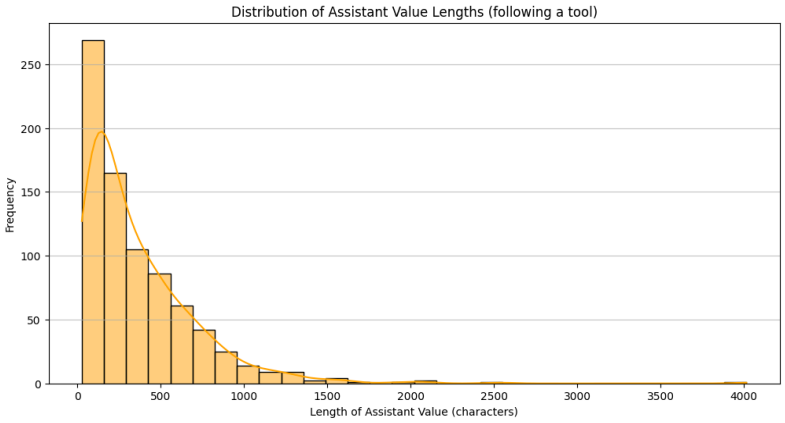
Final Table with all experiments (token-level)

Type	LookBackLens	LettuceDetect F1	Fine-tuned F1	Less ClearData FT F1	Weight Importance FT F1	Hybrid FT
conflict	0.128	0.8000	0.2143	0.2069	0.4571	**0.8000**
overgeneration	0.833	0.8889	0.7671	0.7671	0.9535	**0.9535**
missing_tool	0.888	0.8000	1.0000	1.0000	1.0000	**1.0000**
clean	**0.987**	0.5246	0.9773	0.9535	0.9773	0.9655
overall	0.709	0.7385	0.7876	0.7755	0.8826	**0.9134**

Appendix

A. Initial length distribution statistics





B. Overgeneration prompt

```python
def build_overgeneration_prompt(example: Dict) -> str:
    return (
        "Generate ONE unsupported clause for hallucination evaluation.\n"
        "Overgeneration = same topic as the query, but NOT stated in the tool context.\n\n"
        "GOOD:\n"
        "  the weather has been fairly stable over the past few months\n"
        "  demand for this route has remained strong throughout the season\n\n"
        "BAD (do NOT output these):\n"
        "  good clause: the weather is nice\n"
        "  would you like me to book a hotel\n"
        "  based on the tool output, the price is $50\n"
        "  can you also provide more details?\n"
        "  Stranger Things (ID: 80100172), The Witcher (ID: 80189685)\n\n"
        "Rules:\n"
        "- 6 to 16 words only\n"
        "- lowercase start, no trailing punctuation\n"
        "- no labels, no lists, no IDs, no quotes, no questions\n"
        "- no tool/API mentions, no action offers\n"
        "- return ONLY the clause text\n\n"
        f"Query:\n{example['query'][:600]}\n\n"
        f"Tool context:\n{example['context'][:900]}\n\n"
        f"Assistant response:\n{example['output'][:900]}\n"
    )
```